

Summary and Highlights:

Introduction to Apache Spark

- Spark is an open-source, in-memory application framework for distributed data processing and iterative analysis on massive data volumes. Both distributed systems and Apache Spark are inherently scalable and fault-tolerant.
- Apache Spark solves the problems encountered with MapReduce by keeping a substantial portion of the data required in memory, avoiding expensive and time-consuming disk I/O.
- Functional programming follows a declarative programming model that emphasizes *what* instead of *how-to* and uses expressions.
- Lambda functions or operators are anonymous functions that enable functional programming. Spark parallelizes computations using lambda calculus, and all functional Spark programs are inherently parallel.
- Resilient distributed datasets, or RDDs, are Spark's primary data abstraction consisting of a fault-tolerant collection of elements partitioned across the cluster's nodes, capable of accepting parallel operations.
- You can create an RDD using an external or local Hadoop-supported file, from a collection, or from another RDD. RDDs are immutable and always recoverable, providing resilience in Apache Spark. RDDs can persist or cache datasets in memory across operations, which speeds up iterative operations in Spark.
- Apache Spark architecture consists of components data, compute input, and management. The fault-tolerant Spark Core base engine performs large-scale big data-worthy parallel and distributed data processing jobs, manages memory, schedules tasks, and houses APIs that define RDDs.
- Spark SQL provides a programming abstraction called DataFrames and can also act as a distributed SQL query engine.

- Spark DataFrames are conceptually equivalent to a table in a relational database or a data frame in R or Python, but with richer optimizations.

Package/Method	Description	Code Example
appName()	A name for your job to display on the cluster web UI.	<pre> 1. from pyspark.sql import SparkSession 2. spark = SparkSession.builder.appName("MyApp").getOrCreate() </pre>
cache()	An Apache Spark transformation often used on a DataFrame, data set, or RDD when you want to perform multiple actions. cache() caches the specified DataFrame, data set, or RDD in the memory of your cluster's workers. Since cache() is a transformation, the caching operation takes place only when a Spark action (for example, count(), show(), take(), or write()) is	<pre> 1. df = spark.read.csv("customer.csv") 2. df.cache() </pre>

Package/Method	Description	Code Example
	also used on the same DataFrame, data set, or RDD in a single action.	
count()	Returns the number of elements with the specified value.	<pre> 1. count = df.count() 2. print(count) </pre>
createTempView()	Creates a temporary view that can later be used to query the data. The only required parameter is the name of the view.	<pre> 1. df.createOrReplaceTempView("cust_tbl") </pre>
filter()	Returns an iterator where the items are filtered through a function to test if the item is accepted or not.	<pre> 1. filtered_df = df.filter(df['age'] > 30) </pre>
getOrCreate()	Get or instantiate a SparkContext and register it as a singleton object.	<pre> 1. spark = SparkSession.builder.getOrCreate() </pre>
import	Used to make code from one	<pre> 1. from pyspark.sql import SparkSession </pre>

Package/Method	Description	Code Example
	module accessible in another. Python imports are crucial for a successful code structure. You may reuse code and keep your projects manageable by using imports effectively, which can increase your productivity.	
len()	Returns the number of items in an object. When the object is a string, the len() function returns the number of characters in the string.	<pre> 1. row_count = len(df.collect()) 2. print(row_count) </pre>
map()	Returns a map object (an iterator) of the results after applying the given function to each item of a given iterable (list, tuple, etc.)	<pre> 1. rdd = df.rdd.map(lambda row: (row['name'], 2. row['age'])) </pre>
pip	To ensure that requests will	<pre> 1. pip list </pre>

Package/Method	Description	Code Example
	function, the pip program searches for the package in the Python Package Index (PyPI), resolves any dependencies, and installs everything in your current Python environment.	
pip install	The pip install <package> command looks for the latest version of the package and installs it.	1. pip install pyspark
print()	Prints the specified message to the screen or other standard output device. The message can be a string or any other object; the object will be converted into a string before being written to the screen.	1. print("Hello, PySpark!")
printSchema()	Used to print or display the	1. df.printSchema()

Package/Method	Description	Code Example
	schema of the DataFrame or data set in tree format along with the column name and data type. If you have a DataFrame or data set with a nested structure, it displays the schema in a nested tree format.	
sc.parallelize()	Creates a parallelized collection. Distributes a local Python collection to form an RDD. Using range is recommended if the input represents a range for performance.	1. rdd = sc.parallelize([1, 2, 3, 4, 5])
select()	Used to select one or multiple columns, nested columns, column by index, all columns from the list, by	1. selected_df = df.select('name', 'age')

Package/Method	Description	Code Example
	regular expression from a DataFrame. select() is a transformation function in Spark and returns a new DataFrame with the selected columns.	
show()	Spark DataFrame show() is used to display the contents of the DataFrame in a table row and column format . By default, it shows only twenty rows, and the column values are truncated at twenty characters.	1. df.show()
spark.read.json	Spark SQL can automatically infer the schema of a JSON data set and load it as a DataFrame. The read.json() function loads data from a	1. json_df = spark.read.json("customer.json")

Package/Method	Description	Code Example
	directory of JSON files where each line of the files is a JSON object. Note that the file offered as a JSON file is not a typical JSON file.	
spark.sql()	To issue any SQL query, use the sql() method on the SparkSession instance . All spark.sql queries executed in this manner return a DataFrame on which you may perform further Spark operations if required.	<pre> 1. result = spark.sql("SELECT name, age FROM cust_tbl WHERE age > 30") 2. result.show()</pre>
time()	Returns the current time in the number of seconds since the Unix Epoch.	<pre> 1. from pyspark.sql.functions import current_timestamp 2. current_time = df.select(current_timestamp().alias("current_time")) 3. current_time.show()</pre>